

Lab 2: Higher-Order Functions

- Due: Wednesday 09/09 @ 11:59pm
- Points: 1
- Download: lab02.zip

Attendance

All students will get attendance this week.

If you get stuck, there are walkthrough videos (@berkeley.edu login to YouTube required).

Required Questions

Review

What do you think will happen if we type the following into Python?

```
1 / 0
```

Try it out in Python! You should see a ZeroDivisionError. But what about this expression?

```
True or 1 / 0
```

It evaluates to True because Python's and and or operators short-circuit. That is, they don't necessarily evaluate every operand.

Operator Checks if: Evaluates from left to right up to: Example

AND All values are true The first false value False and 1 / 0 evaluates to False

OR At least one value is true The first true value True or 1 / 0 evaluates to True

Short-circuiting happens when the operator reaches an operand that allows them to make a conclusion about the expression. For example, and will short-circuit as soon as it reaches the first false value because it then knows that not all the values are true.

If and and or do not short-circuit, they just return the last value; another way to remember this is that and and or always return the last thing they evaluate, whether they short-circuit or not. Keep in mind that and and or don't always return booleans when using values other than True and False.

Lambda expressions are expressions that evaluate to functions by specifying two things: the parameters and a return expression.

```
lambda <parameters>: <return expression>
```

While both lambda expressions and def statements create function objects, there are some notable differences. lambda expressions work like other expressions; much like a mathematical expression just evaluates to a number and does not alter the current environment, a lambda expression evaluates to a function without changing the current environment.

lambda def

Type An expression that evaluates to a value. A statement that alters the environment.

Result of execution Creates an anonymous lambda function with no intrinsic name. Creates a function with an intrinsic name and binds it to that name in the current environment.

Effect on the environment Evaluating a lambda expression does not create or modify any variables. Executing a def statement both creates a new function object and binds it to a name in the current environment.

Usage A lambda expression can be used anywhere that expects an expression, such as in an assignment statement or as the operator or operand of a call expression. After executing a def statement, use the function's bound name anywhere that expects an expression.

lambda examples

```
# A lambda expression by itself does not alter
# the environment
lambda x: x * x
# We can assign lambda functions to a name
```

```
# with an assignment statement
square = lambda x: x * x
square(3)

# Lambda expressions can be used as an operator
# or operand
negate = lambda f, x: -f(x)
negate(lambda x: x * x, 3)
```

def example

```
def square(x):
    return x * x

# A function created by a def statement
# can be referred to by its intrinsic name
square(3)
```

Variables are names bound to values, which can be primitives like 3 or 'Hello World', but they can also be functions. And since functions can take arguments of any value, other functions can be passed in as arguments. This is the basis for higher-order functions.

A higher-order function is a function that manipulates other functions by taking in functions as arguments, returning a function, or both.

Functions as arguments

In Python, function objects are values that can be passed around. We know that one way to create functions is by using a def statement:

```
def square(x):
    return x * x
```

The above statement created a function object with the intrinsic name square as well as binded it to the name square in the current environment. Now let's try passing it as an argument.

First, let's write a function that takes in another function as an argument:

```
def scale(f, x, k):
    """ Returns the result of f(x) scaled by k. """
    return k * f(x)
```

We can now call scale on square and some other arguments:

```
>>> scale(square, 3, 2) # Double square(3)
18
>>> scale(square, 2, 5) # 5 times 2 squared
20
```

Note that in the body of the call to scale, the function object with the intrinsic name square is bound to the parameter f. Then, we call square in the body of scale by calling f(x).

As we saw in the section on lambda expressions, we can also pass lambda expressions into call expressions!

```
>>> scale(lambda x: x + 10, 5, 2)
30
```

In the frame for this call expression, the name f is bound to the function created by the lambda expression lambda x: x + 10.

Functions that return functions

Because functions are values, they are valid as return values! Here's an example:

```
def multiply_by(m):
    def multiply(n):
        return n * m
    return multiply
```

In this particular case, we defined the function multiply within the body of multiply_by and then returned it. Let's see it in action:

```
>>> multiply_by(3)
<function multiply_by.<locals>.multiply at ...>
>>> multiply(4)
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
NameError: name 'multiply' is not defined
```

A call to `multiply_by` returns a function, as expected. However, calling `multiply` errors, even though that's the name we gave the inner function. This is because the name `multiply` only exists within the frame where we evaluate the body of `multiply_by`.

So how do we actually use the inner function? Here are two ways:

```
>>> times_three = multiply_by(3) # Assign the result of the call expression to a name
>>> times_three(5) # Call the inner function with its new name
15
>>> multiply_by(3)(10) # Chain together two call expressions
30
```

The point is, because `multiply_by` returns a function, you can use its return value just like you would use any other function.

Environment diagrams are one of the best learning tools for understanding lambda expressions and higher-order functions because you're able to keep track of all of the different names, function objects, and arguments to functions. We highly recommend drawing environment diagrams or using Python tutor if you get stuck doing the WWPD problems below. For examples of what environment diagrams should look like, try running some code in Python tutor. Here are the rules:

Assignment Statements

- Evaluate the expression on the right hand side of the `=` sign.
- If the name found on the left hand side of the `=` doesn't already exist in the current frame, write it in. If it does, erase the current binding. Bind the value obtained in step 1 to this name.

If there is more than one name/expression in the statement, evaluate all of the expressions first from left to right before making any bindings.

def Statements

- Draw the function object with its intrinsic name, formal parameters, and parent frame. A function's parent frame is the frame in which the function was defined.
- If the intrinsic name of the function doesn't already exist in the current frame, write it in. If it does, erase the current binding. Bind the newly created function object to this name.

Call expressions

Note: you do not have to go through this process for a built-in Python function like `max` or `print`.

- Evaluate the operator, whose value should be a function.
- Evaluate the operands left to right.
- Open a new frame. Label it with the sequential frame number, the intrinsic name of the function, and its parent.
- Bind the formal parameters of the function to the arguments whose values you found in step 2.
- Execute the body of the function in the new environment.

Lambdas

Note: As we saw in the lambda expression section above, lambda functions have no intrinsic name. When drawing lambda functions in environment diagrams, they are labeled with the name `lambda` or with the lowercase Greek letter λ . This can get confusing when there are multiple lambda functions in an environment diagram, so you can distinguish them by numbering them or by writing the line number on which they were defined.

- Draw the lambda function object and label it with λ , its formal parameters, and its parent frame. A function's parent frame is the frame in which the function was defined.

This is the only step. We are including this section to emphasize the fact that the difference between lambda expressions and `def` statements is that lambda expressions do not create any new bindings in the environment.

What Would Python Display? (WWPD)

Q1: WWPDP: The Truth Will Prevail

Predict what Python will display when the following lines are entered into an interactive session, then unlock the test to check your answers. Type FUNCTION if you believe the answer is a function, ERROR if it errors, and NOTHING if nothing is displayed:

```
python3 -m pytest -k short_circuit --unlock
```

```
>>> True and 13
_____
>>> False or 0
_____
>>> not 10
_____
>>> not None
_____

>>> True and 1 / 0
_____
>>> True or 1 / 0
_____
>>> -1 and 1 > 0
_____
>>> -1 or 5
_____
>>> (1 + 1) and 1
_____
>>> print(3) or ""
_____
_____

>>> def f(x):
...     if x == 0:
...         return "zero"
...     elif x > 0:
...         return "positive"
...     else:
...         return ""
>>> 0 or f(1)
_____
>>> f(0) or f(-1)
_____
>>> f(0) and f(-1)
_____
```

Q2: WWPDP: Higher-Order Functions

Predict what Python will display when the following lines are entered into an interactive session, then unlock the test to check your answers:

Important: Type FUNCTION if you believe the answer is a function value, such as <function f at ...> or <built-in function pow>.

```
python3 -m pytest -k hof_wwpd --unlock
```

```
>>> def cake():
...     print('beets')
...     def pie():
...         print('sweets')
...         return 'cake'
...     return pie
>>> chocolate = cake()
_____
>>> chocolate
_____
>>> chocolate()
_____

>>> more_chocolate, more_cake = chocolate(), cake
_____
>>> more_chocolate
_____

>>> def snake(x, y):
...     if cake == more_cake:
...         return chocolate
...     else:
```

```

...         return x + y
>>> snake(10, 20)

_____
>>> snake(10, 20)()

_____
>>> cake = 'cake'
>>> snake(10, 20)

_____

```

Q3: WWPDP: Lambda

Predict what Python will display when the following lines are entered into an interactive session, then unlock the test to check your answers:

Important: Type FUNCTION if you believe the answer is a function value. As a reminder, the following two lines of code will not display any output in the interactive Python interpreter when executed:

```

>>> x = None
>>> x
>>>

python3 -m pytest -k lambda_wwpd --unlock

>>> lambda x: x # A lambda expression with one parameter x

_____
>>> a = lambda x: x # Assigning the lambda function to the name a
>>> a(5)

_____
>>> (lambda: 3)() # Using a lambda expression as an operator in a call expression

_____
>>> b = lambda x, y: lambda: x + y # Lambdas can return other lambdas!
>>> c = b(8, 4)
>>> c

_____
>>> c()

_____
>>> d = lambda f: f(4) # They can have functions as arguments as well
>>> def square(x):
...     return x * x
>>> d(square)

_____

>>> higher_order_lambda = lambda f: lambda x: f(x)
>>> g = lambda x: x * x
>>> higher_order_lambda(g)(2)

_____
>>> call_thrice = lambda f: lambda x: f(f(f(x)))
>>> call_thrice(lambda y: y + 2)(5)

_____
>>> print_lambda = lambda z: print(z) # When is the return expression of a lambda expression
executed?
>>> print_lambda

_____
>>> one_thousand = print_lambda(1000)

_____
>>> print(one_thousand) # What did the call to print_lambda return?

_____

```

Write Code

Q4: Piecewise

Implement piecewise, which takes two one-argument functions, f and g, along with a number b. It returns a new function that takes a number x and returns either f(x) if x is less than b, or g(x) if x is greater than or equal to b.

```

def piecewise(f, g, b):
    """Returns the piecewise function h where:

        h(x) = f(x) if x < b,
               g(x) otherwise

    >>> def negate(x):
    ...     return -x

```

```

>>> identity = lambda x: x
>>> abs_value = piecewise(negate, identity, 0)
>>> abs_value(6)
6
>>> abs_value(-1)
1
"""
**** YOUR CODE HERE ****

python3 -m pytest -k piecewise

```

Q5: Count Cond

Consider the following implementations of `count_fives` and `count_primes`, which use the `sum_digits` and `is_prime` functions defined in your starter file:

```

def count_fives(n):
    """Return the number of values i from 1 to n (including n)
    where sum_digits(n * i) is 5.

    >>> count_fives(10) # Among 10, 20, 30, ..., 100, only 50 (10 * 5) has digit sum 5
    1
    >>> count_fives(50) # 50 (50 * 1), 500 (50 * 10), 1400 (50 * 28), 2300 (50 * 46)
    4
    """
    i = 1
    count = 0
    while i <= n:
        if sum_digits(n * i) == 5:
            count += 1
        i += 1
    return count

def count_primes(n):
    """Return the number of prime numbers up to and including n.

    >>> count_primes(6) # 2, 3, 5
    3
    >>> count_primes(13) # 2, 3, 5, 7, 11, 13
    6
    """
    i = 1
    count = 0
    while i <= n:
        if is_prime(i):
            count += 1
        i += 1
    return count

```

The implementations look quite similar! Generalize this logic by writing a function `count_cond`, which takes in a two-argument predicate function `condition(n, i)`. `count_cond` returns a one-argument function that takes in `n`, which counts all the numbers from 1 to `n` that satisfy `condition` when called.

Note: When we say `condition` is a predicate function, we mean that it is a function that will return `True` or `False`.

```

def count_cond(condition):
    """Returns a function with one parameter n that counts all the numbers i
    (1 to n) that satisfy the two-argument predicate function condition, where
    the first argument for condition is n and the second argument is i.

    >>> count_fives = count_cond(lambda n, i: sum_digits(n * i) == 5)
    >>> count_fives(10) # 50 (10 * 5)
    1
    >>> count_fives(50) # 50 (50 * 1), 500 (50 * 10), 1400 (50 * 28), 2300 (50 * 46)
    4

    >>> is_i_prime = lambda n, i: is_prime(i) # need to pass 2-argument function into
count_cond
    >>> count_primes = count_cond(is_i_prime)
    >>> count_primes(2) # 2
    1
    >>> count_primes(3) # 2, 3
    2
    >>> count_primes(4) # 2, 3
    2

```

```

>>> count_primes(5)    # 2, 3, 5
3
>>> count_primes(20)   # 2, 3, 5, 7, 11, 13, 17, 19
8
"""
**** YOUR CODE HERE ****
python3 -m pytest -k count_cond

```

Environment Diagrams

Q6: HOF Diagram Practice

Draw the environment diagram that results from executing the code below on paper or a whiteboard. Use tutor.cs61a.org to check your work. There is nothing to submit for this question.

```

n = 7

def f(x):
    n = 8
    return x + 1

def g(x):
    n = 9
    def h():
        return x + 1
    return h

def f(f, x):
    return f(x + n)

f = f(g, n)
g = (lambda y: y())(f)

```

Survey

Q7: Week 3 Survey

As part of this assignment, fill out the Week 3 Survey form.

Once you finish the survey, you will be presented with a passphrase. Put this passphrase, as a string, on the line that says `passphrase = 'REPLACE_THIS_WITH_PASSPHRASE'` in the Python file for this assignment. E.g. if the passphrase is `abc`, then the line should be `passphrase = 'abc'`.

```
python3 -m pytest -k week3_survey
```

Submit Assignment

Submit this assignment by running Provenance: Prepare Submission Bundle from the VS Code command palette and uploading the resulting zip to Gradescope. The zip already contains the files you've edited. Lab 00 has detailed instructions.

Your responses to WWPd questions are not submitted, and they do not need to be. Lab credit is based on the code writing questions.

Optional Questions

These questions are optional. If you don't complete them, you will still receive credit for this assignment. They are great practice, so do them anyway!

Q8: I Heard You Liked Functions...

Define a function `cycle` that takes in three functions `f1`, `f2`, and `f3`, as arguments. `cycle` will return another function `g` that should take in an integer argument `n` and return another function `h`. That final function `h` should take in an argument `x` and cycle through applying `f1`, `f2`, and `f3` to `x`, depending on what `n` was. Here's what the final function `h` should do to `x` for a few values of `n`:

- `n = 0`, return `x`
- `n = 1`, apply `f1` to `x`, or return `f1(x)`

- $n = 2$, apply f_1 to x and then f_2 to the result of that, or return $f_2(f_1(x))$
- $n = 3$, apply f_1 to x , f_2 to the result of applying f_1 , and then f_3 to the result of applying f_2 , or $f_3(f_2(f_1(x)))$
- $n = 4$, start the cycle again applying f_1 , then f_2 , then f_3 , then f_1 again, or $f_1(f_3(f_2(f_1(x))))$
- And so forth.

Hint: most of the work goes inside the most nested function.

Hint: How can you utilize the $\%$ operator to achieve the cyclic behavior? Try computing $n \% 3$ for all integers n from 0 to 12. What pattern do you notice?

```
def cycle(f1, f2, f3):
    """Returns a function that is itself a higher-order function.

    >>> def add1(x):
    ...     return x + 1
    >>> def times2(x):
    ...     return x * 2
    >>> def add3(x):
    ...     return x + 3
    >>> my_cycle = cycle(add1, times2, add3)
    >>> identity = my_cycle(0)
    >>> identity(5)
    5
    >>> add_one_then_double = my_cycle(2)
    >>> add_one_then_double(1)
    4
    >>> do_all_functions = my_cycle(3)
    >>> do_all_functions(2)
    9
    >>> do_more_than_a_cycle = my_cycle(4)
    >>> do_more_than_a_cycle(2)
    10
    >>> do_two_cycles = my_cycle(6)
    >>> do_two_cycles(1)
    19
    """
    """ YOUR CODE HERE """
```

```
python3 -m pytest -k cycle
```